เข้าใจภาพรวมก่อนลงมือ

API พื้นฐานด้วย Go Fiber v2 และ validator/v10 | อ่านตามลำดับแล้วลองทำได้ทันที

โปรเจกต์นี้ทำอะไรได้บ้าง

5.0 ตรวจ Basic Auth ก่อนเข้าใช้งานทุก API
5.1 รับเลขจาก URL แล้วคำนวณ factorial เช่น 5! = 120
5.2 รับ tax_id จาก Query แล้วแปลงตัวอักษรเป็นรหัส ASCII
5.3 ใช้ชื่อย่อ c แทน controller เพื่อเรียกฟังก์ชันให้สั้นลง
6 รับข้อมูลสมัครสมาชิกและตรวจความถูกต้องด้วย validator

หนึ่ง request เดินทางอย่างไร

```
Postman -> Basic Auth -> Route -> Controller -> Response
```

Request คือคำขอที่ส่งเข้า API เช่น ขอคำนวณหรือส่งข้อมูลสมัครสมาชิก
Basic Auth ตรวจชื่อผู้ใช้และรหัสผ่าน ถ้าผิดจะหยุดและตอบ 401
Route เลือกฟังก์ชันจาก method และ URL ส่วน Controller ทำงานนั้น
Response คือผลที่ส่งกลับ เช่น HTTP 200 พร้อมข้อมูล JSON

แต่ละไฟล์มีหน้าที่เดียว อ่านตามนี้ได้เลย

ไฟล์ / โฟลเดอร์	หน้าที่
main.go	สร้าง app และเปิด server ที่ port 3000
routes/routes.go	ตั้ง Basic Auth, group และเส้นทาง API
controller/	รับข้อมูล ประมวลผล และตอบกลับ
models/register.go	กำหนดรูปแบบข้อมูลและกฎ validate
routes/routes_test.go	ทดสอบ API แบบอัตโนมัติ

ขอบเขตข้อ 6: ตัวอย่างนี้ตรวจข้อมูลเท่านั้น ยังไม่มีการบันทึกสมาชิก
ฟิลด์ตัวอย่าง: username, email, password และ confirm_password

เริ่มรันและผ่าน Basic Auth

เตรียม server ให้ทำงาน แล้วตั้งค่า Postman ก่อนทดสอบทุก endpoint

1. เปิด terminal ที่โฟลเดอร์โปรเจกต์

```
cd C:\src\workshop-go
go version
go mod download
go run .
```

ใช้ Go 1.26 ขึ้นไปตาม go.mod ของโปรเจกต์นี
go mod download ดาวน์โหลดไลบรารี ส่วน go run . เริ่มโปรแกรม
เปิด terminal นี้ค้างไว้ขณะทดสอบ หยุด server ด้วย Ctrl+C
ถ้าแก้โค้ด ให้หยุดแล้วสั่ง go run . ใหม่เพื่อใช้โค้ดที่แก้

2. เรียก API แรกใน Postman

สร้าง HTTP request แล้วเลือก method เป็น GET
ใส่ URL: http://localhost:3000/api/v1/fact/5
ที่ Authorization เลือกชนิด Basic Auth แล้วกรอกค่าด้านล่าง

ช่อง	ค่าที่ต้องกรอก
Username	gofiber
Password	21022566

กด Send ควรเห็นสถานะ 200 OK และผล factorial เท่ากับ 120
ตั้ง Basic Auth แบบเดียวกันในทุก request รวมถึง POST register

3. ทดลองดูว่าระบบป้องกัน API จริง

เปลี่ยน Password เป็น wrong แล้วกด Send ต้องได้ 401 Unauthorized
เลือก No Auth แล้วส่งอีกครั้ง ต้องได้ 401 เช่นกัน
เมื่อทดสอบเสร็จ ให้กลับมาใช้ Basic Auth และรหัสผ่านที่ถูกต้อง

> Basic Auth ชุดนี้ใช้เข้าถึง API ตามโจทย์ ส่วน password ใน body สมัครสมาชิก
> เป็นข้อมูลของสมาชิก ทั้งสองค่าอยู่คนละส่วนและทำหน้าที่ต่างกัน

Path, Query และการจัด routes

ข้อ 5.1 - 5.3 | ทุกตัวอย่างในหน้านี้ต้องส่ง Basic Auth ทีถูกต้อง

Path parameter: รับเลข factorial

```
GET http://localhost:3000/api/v1/fact/5

{"number":5,"factorial":120}
```

เลข 5 อยู่ใน path ตรงตำแหน่ง :number ของ route
ctx.Params("number") อ่านเป็นข้อความ แล้ว strconv.Atoi แปลงเป็น int
เรีม result = 1 แล้ววนคูณ 1, 2, 3, 4, 5 จนได้ 120
รับจำนวนเต็ม 0 ถึง 20 โดย 0! = 1 และ 21! เกินขนาด int64

Query parameter: รับ tax_id แล้วแปลง ASCII

```
GET http://localhost:3000/api/v3/joe?tax_id=Hello

{"tax_id":"Hello","ascii":"72 101 108 108 111"}
```

ใน Postman ใช้แท็บ Params: Key = tax_id และ Value = Hello
ctx.Query("tax_id") อ่านค่าหลังเครื่องหมาย ? ใน URL
วนทีละตัวอักษร: H = 72, e = 101, l = 108, l = 108, o = 111
แปลงตัวเลขเป็นข้อความ แล้ว strings.Join รวมโดยคันด้วยช่องว่าง
รับตัวเลขและอักขระ ASCII ได้ ภาษาไทยอยู่นอกช่วง ASCII จึงตอบ 400

Group และชื่อย่อ c อยู่ใน routes/routes.go

```
import c "workshop-go/controller"

v3 := app.Group("/api/v3")
v3.Get("/joe", c.TestParams)
```

Group รวม prefix กับ path ย่อย จึงได้ URL เต็มเป็น /api/v3/joe
เปลียน /joe เป็นชื่อเล่นของตัวเอง แล้วเรีม server ใหม่
c เป็นชื่อย่อเฉพาะตอน import จึงเรียก c.TestParams ได้

จำง่าย: Path = /fact/5 | Query = ?tax_id=Hello
Body = ข้อมูล JSON ทีจะส่งใน POST สมัครสมาชิกในหน้าถัดไป

ลองสมัครสมาชิกด้วย Postman

ข้อ 6 | ตัวอย่างนีรับ JSON และใช้ validator ตรวจสอบข้อมูลทัง 4 ฟิลด์

เลือก POST และ URL: http://localhost:3000/api/v1/register
ตัง Authorization เป็น Basic Auth ตามหน้า 2
เลือก Body > raw > JSON และตรวจ Content-Type: application/json

ข้อมูลทีส่งใน body

```
{
  "username": "student01",
  "email": "student@example.com",
  "password": "password123",
  "confirm_password": "password123"
}
```

ฟิลด์	กฎทีใช้ในโปรเจกต์
username	จำเป็น, A-Z / a-z / 0-9 เท่านัน, ยาว 3-20 ตัว
email	จำเป็น, รูปแบบอีเมลถูกต้อง, ไม่เกิน 254 ตัว
password	จำเป็น, ความยาว 8-72 ตัว
confirm_password	จำเป็น และต้องตรงกับ password

กด Send: ข้อมูลผ่านจะได้ HTTP 200

```
{
 "message": "ข้อมูลสมัครสมาชิกถูกต้อง",
  "data": {"username":"student01",
           "email":"student@example.com"}
}
```

ลองเปลียน email เป็น abc แล้วส่งอีกครัง

```
HTTP 400 Bad Request
{"message":"ใส่ข้อมูลผิดพลาด"}
```

ตอบสำเร็จแปลว่าข้อมูลผ่านกฎ ยังไม่ได้สร้างบัญชีในฐานข้อมูล
ผลตอบกลับจะไม่ส่ง password หรือ confirm_password กลับมา

เข้าใจ validator ทีละขั้น

อ่าน models/register.go แล้วตามด้วย controller/register.go

1. Struct คือแบบฟอร์มที Go ใช้เก็บข้อมูล

```
Email string `json:"email" validate:"required,email,max=254"`
```

Email คือชื่อฟิลด์ใน Go และ string คือชนิดข้อมูลข้อความ
json:"email" บอกว่าให้รับค่าจาก key ชื่อ email ใน JSON
validate กำหนดกฎคัดด้วยจุลภาค ต้องผ่านทุกกฎที่ระบุ

Tag	ความหมาย
required	ต้องมีค่า ไม่ใช่ข้อความว่าง
alphanum	ตัวอักษร A-Z, a-z หรือตัวเลข 0-9
min=3 / max=20	ข้อความยาวอย่างน้อย 3 และไม่เกิน 20 ตัว
email	ตรวจรูปแบบอีเมล ไม่ได้ตรวจว่ากล่องอีเมลมีจริง
eqfield=Password	ค่าต้องตรงกับฟิลด์ชื่อ Password ใน Go

2. BodyParser อ่านข้อมูลเข้า struct

```
var input models.RegisterRequest
err := ctx.BodyParser(&input)
```

&input คือส่งที่อยู่ของตัวแปร เพื่อให้ BodyParser ใส่ค่าลงไปได้
ถ้า JSON พัง หรือส่งตัวเลขให้ฟิลด์ string จะเกิด error แล้วตอบ 400
อีเมล abc ยังเป็น string จึงอ่านได้ แต่ยังไม่ผ่านกฎ email

3. validate.Struct ตรวจว่าค่าผ่านกฎหรือไม่

```
err := validate.Struct(input)
// nil = passed; non-nil = failed
```

ถ้ามี error ให้ return ข้อความผิดพลาดทันที จึงไม่ทำขั้นตอนถัดไป
ถ้าไม่มี error จึงส่ง username และ email กลับเป็น JSON

BodyParser = อ่านข้อมูล | validator = ตรวจค่าตามกฎ
สร้าง validate ครั้งเดียว และใช้ tags สำเร็จรูปแทนการเขียน regexp เอง

ตารางทดสอบด้วยตัวเอง

เปลี่ยนทีละค่าแล้วเทียบผล | กลับไปใช้ข้อมูลถูกต้องก่อนเริ่มกรณีถัดไป

ทดสอบ Basic Auth และ API ข้อ 5

กรณี	ค่าทีลอง	ผลทีควรได้
Auth	No Auth หรือรหัสผิด	401
Factorial	/fact/5	200, factorial = 120
Factorial	/fact/0	200, factorial = 1
Factorial	/fact/20	200, คำนวณได้
Factorial	/fact/-1 หรือ /fact/21	400
Factorial	/fact/abc หรือ /fact/1.5	400
ASCII	tax_id=Hello	200, 72 101 108 108 111
ASCII	tax_id=A12	200, 65 49 50
ASCII	ไม่ส่ง tax_id / ว่าง / ภาษาไทย	400

ทดสอบ register จาก JSON ตัวอย่างหน้า 4

แก้ข้อมูลอย่างไร	ผลทีควรได้
ส่งข้อมูลตัวอย่างครบและถูกต้อง	200
ส่ง {} หรือลบ confirm_password ออก	400
username เป็น ab หรือ student!	400
email เป็น abc	400
password และ confirm_password เป็น short	400
confirm_password เป็น different	400
username เป็น 123 แบบไม่มีเครื่องหมายคำพูด	400
ลบวงเล็บปีกกาปิด } ทำให้ JSON ไม่ครบ	400

กรณีข้อมูลสมัครสมาชิกผิดในตาราง ต้องได้ message: ใส่ข้อมูลผิดพลาด
ใช้ Basic Auth ทีถูกต้องก่อน มิฉะนันจะได้ 401 ก่อนถึงการตรวจ body

ทดสอบอัตโนมัติและแก้ปัญหา

ทบทวนความเข้าใจ แล้วทดลองแก้โค้ดเล็กน้อยด้วยตัวเอง

รันชุดทดสอบที่เตรียมไว้

```
go test ./...
go test ./routes -v
```

คำสั่งแรกทดสอบทุก package ส่วน -v แสดงชื่อแต่ละกรณีที่ทดสอบ
ไม่ต้องเปิด go run . เพราะชุดทดสอบใช้ app.Test เรียกแอปภายใน
ดูคำว่า ok หรือ PASS ถ้าล้มเหลวจะมี FAIL และรายละเอียดที่ผิด
รันตรวจสอบก่อนจัดทำคู่มือนี้แล้ว: go test ./... ผ่าน
ครอบคลุม 33 กรณีย่อย ทั้ง Auth, factorial, ASCII และ register

เจออาการนี้ ให้ตรวจตรงไหน

อาการ	สิ่งที่ควรตรวจ
Connection refused	เปิด server ด้วย go run . และใช้ port 3000
Port 3000 ถูกใช้อยู่	หยุด server เดิมใน terminal ด้วย Ctrl+C
401 Unauthorized	ตรวจ Basic Auth: gofiber / 21022566
404 Not Found	ตรวจ method, URL และชื่อเล่นใน routes
400 ที่ register	ตรวจ JSON, ชนิดข้อมูล และกฎทั้ง 4 ฟิลด์
แก้โค้ดแล้วผลเหมือนเดิม	หยุดแล้วรัน go run . ใหม่

แบบฝึกหัดสั้น ๆ

1. เปลี่ยน /joe เป็นชื่อเล่นตัวเอง แล้วลอง URL ใหม่
2. เปลี่ยน username จาก min=3 เป็น min=5 แล้วลองชื่อยาว 4 ตัว
3. อธิบายว่าเหตุใด email = abc อ่าน body ได้ แต่ validate ไม่ผ่าน
4. ลอง No Auth กับข้อมูลถูกต้อง แล้วอธิบายว่าทำไมได้ 401

จุดอ่านต่อและแหล่งอ้างอิง

อ่าน README.md สำหรับตัวอย่าง และ routes/routes_test.go สำหรับ test
คู่มือนี้อ้างอิงโค้ดใน C:\src\workshop-go ณ วันที่จัดทำ

validator: https://github.com/go-playground/validator

Fiber v2: https://pkg.go.dev/github.com/gofiber/fiber/v2